

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY
DATABASE MANAGEMENT SYSTEMS**



**GROUP PROJECT REPORT - GROUP 6
F1 CHAMPIONSHIP MANAGEMENT SYSTEM**

Supervisor: Assoc. Prof. Dr. Nguyen Trong Khanh
Team members: Nguyen Van Hieu - B23DCCE033
Pham Nhat Minh - B23DCCE066
Le Minh Duc - B23DCVT095

Hanoi, April 2026

Contents

1	Problem Introduction	6
1.1	Context	6
1.2	System Objectives	6
1.3	Functional Requirements (FR)	6
1.4	Core Business Rules	7
2	Configuration, Tools, and Platform	8
2.1	Hardware Environment and Deployment Configuration	8
2.2	Technologies Used	9
2.3	Architecture Rationale	9
2.4	Database Server Configuration and Optimization	9
2.4.1	RAM and Connection Tuning	9
2.4.2	Hardware Resource Exploitation (CPU & SSD)	10
2.4.3	Auditing, Logging, and Resource Cleanup	10
2.4.4	Automated Database Backup Strategy (Backup Autopilot)	11
2.4.5	Load Testing and System Validation	11
3	Installation & Deployment	13
3.1	Database Design and Normalization	13
3.1.1	Relational Schema	13
3.1.2	Normalization Assessment	15
3.2	Data Definition Language (DDL) Script	16
3.3	Database Triggers for Business Logic	18
3.3.1	Limiting the Number of Drivers per Team	18
3.3.2	Time Validity Check	19
3.3.3	Other Business Integrity Triggers	19
3.4	Stored Procedure and Transaction	22
3.4.1	Operational Stored Procedures	24
3.5	Views and Indexing	26
3.6	Deployment Guide	29
3.6.1	Step 1: Initialize the Database	29
3.6.2	Step 2: Configure Backend	29
3.6.3	Step 3: Launch Frontend	29
3.7	Results	30
4	Conclusion	31
4.1	Summary of Achievements	31
4.2	Lessons Learned	31
4.3	Future Development	31
	References	33

List of Figures

3.1	Entity-Relationship Diagram (ERD)	15
3.2	Processing flow of Trigger trg_limit_2_riders	18
3.3	Transaction flow of Module 2 — Result Entry	23
3.4	F1 Championship Management System interface	30

List of Tables

2.1	Hardware configuration of the development environment	8
2.2	Technology and platform overview	9

Listings

2.1	Automated backup script — auto_backup.bat	11
2.2	Artillery load test configuration — test-tai.yml	12
3.1	DDL — Table creation and constraint setup	16
3.2	Trigger — trg_limit_2_riders	18
3.3	Trigger — trg_check_time_insert	19
3.4	Triggers — Data integrity validation triggers	19
3.5	Stored Procedure sp_calculate_points — Automatic F1 point calculation	23
3.6	Stored Procedures — Driver management and query procedures	24
3.7	Views — All 10 views for reporting and queries	26
3.8	Performance optimization with Indexes	28

Database Terminology Glossary

Term	Definition
ACID	A set of four properties (Atomicity, Consistency, Isolation, Durability) that guarantee the reliability of database transactions.
MVCC	(Multi-Version Concurrency Control) A mechanism that supports concurrent access by allowing multiple versions of data to exist simultaneously.
DBA	(Database Administrator) The administrator responsible for operating, securing, and optimizing the database system.
InnoDB engine	The default storage engine for MySQL that supports ACID transactions, row-level locking, and crash recovery features.
B+ Tree	A self-balancing tree data structure that optimizes search and block-based data retrieval in DBMS.
Buffer pool	A RAM memory area used to cache data and indexes, minimizing direct disk read/write operations.
Redo log	A log that records data changes to restore system state in the event of an unexpected failure.
Undo log	A log that records the previous state of data to support rollback operations and the MVCC mechanism.
Trigger	A SQL code object that automatically executes when an INSERT, UPDATE, or DELETE event occurs on a table.
Transaction	A logical unit of work consisting of a sequence of SQL operations, ensuring either complete execution or no execution at all.
Stored Procedure	A set of precompiled SQL statements stored on the server to execute recurring business operations.
Views	A virtual table that presents data from one or more tables through a predefined query statement.
Index	A data structure that speeds up record lookup, similar to the index of a book.
JWT Authentication	(JSON Web Token) A secure authentication method between system components using encoded tokens.
CTE	(Common Table Expression) A temporary result set that makes complex SQL statements clearer and easier to manage.

Chapter 1

Problem Introduction

1.1 Context

Formula 1 (F1) is the world's premier open-wheel racing championship. With dozens of teams, over 20 professional drivers, and a race calendar spanning nearly an entire year, managing the data lifecycle of this global sport demands precision and high data integrity. Each season comprises multiple races (Grand Prix) held across different countries.

The F1 Championship Management System is a comprehensive software platform designed to manage data across multiple seasons. Before having a centralized DBMS, managing driver contracts, team assignments, lap completion times, and scoring was highly error-prone. This system leverages the Relational Database Management System (RDBMS) model to provide a centralized, consistent, and highly available data repository.

1.2 System Objectives

The system is built to automate the core operations of an F1 season, specifically:

- Manage information about teams, drivers, and their active contracts.
- Schedule races and manage Grand Prix events across seasons.
- Register drivers for specific races based on strict business rules.
- Record race results in real-time and automatically calculate points according to the international FIA standard.
- Provide accurate and continuously updated driver and constructor standings.

1.3 Functional Requirements (FR)

The system must fulfill the following functional requirements:

- FR-01** Manage F1 season information (Create, Update, Read).
- FR-02** Manage the list of teams and drivers, including contract history.
- FR-03** Allow driver registration for each race, with a constraint of a maximum of 2 drivers per team per race.
- FR-04** Allow entry of race results (finish time, laps completed, status).
- FR-05** Automatically calculate points according to the F1 standard (P1=25, P2=18, P3=15, etc.) immediately after saving results.
- FR-06** Provide driver and constructor standings, with filtering support by race history.
- FR-07** Support race result adjustments after saving, with automatic score recalculation.

1.4 Core Business Rules

The database logic is designed to strictly enforce the following rules:

1. **Driver limit:** A team is not allowed to register more than two drivers in the same race.
2. **Scoring system:** Points are calculated based on time ranking. Only drivers who finish the race (`status = 'Finished'`) are eligible for points.
3. **Time validity:** The finish time (`end_time`) must be greater than the race start time.
4. **Contract management:** When a driver transfers to another team, the old contract is set to `is_active = 0`, and a new contract is created with `is_active = 1`.
5. **Constructor points:** The total points of a team at a race equals the sum of points of all drivers belonging to that team at the race.

Chapter 2

Configuration, Tools, and Platform

During the development of the F1 Championship Management System, the selection of hardware, platform technologies, and software architecture plays a decisive role in the system's performance, data integrity, and scalability. Below is a detailed analysis of the environment and tools used by the team.

2.1 Hardware Environment and Deployment Configuration

The system was developed, tested, and load-tested on a personal computer serving as a local server. The hardware configuration not only supports running the source code but must also meet the demanding I/O requirements of the DBMS.

Table 2.1: Hardware configuration of the development environment

Component	Details and Rationale
Operating System	Windows 11 Pro 64-bit / Linux Ubuntu 22.04 LTS. Provides a stable environment for running MySQL and Node.js background services.
Processor (CPU)	Intel Core i5 12th Gen or equivalent. Ensures smooth processing of complex queries containing Window Functions and Subqueries.
Memory (RAM)	8 GB - 16 GB DDR4. Significance for DBMS: Provides sufficient capacity for the InnoDB Buffer Pool, enabling data and index caching in RAM, minimizing disk I/O.
Storage	256 GB SSD (NVMe preferred). Significance for DBMS: The high random write speed of SSDs is crucial for fast Redo Log and Undo Log writes, ensuring Transaction Durability.

2.2 Technologies Used

The application uses a modern technology stack, clearly separated via the 3-Tier Architecture model.

Table 2.2: Technology and platform overview

Tier	Technology	Version	Role
Database	MySQL / MariaDB	8.0 / 10.6+	Relational DBMS
Backend	Node.js	18.x LTS	JavaScript runtime environment
Backend	Express.js	4.x	RESTful API framework
DB Driver	mysql2/promise	3.x	Async Node.js-MySQL connector
Frontend	React.js (Vite)	18.x	SPA UI rendering library
Frontend	TailwindCSS	3.x	Utility-first CSS framework
DB Admin	MySQL Workbench	8.x	DB design and query tool

2.3 Architecture Rationale

The system is designed following the **3-Tier Architecture**:

1. **Presentation Tier (React.js):** Provides a smooth Single Page Application (SPA) experience. Vite is used to leverage extremely fast build speeds (HMR).
2. **Application Tier (Node.js/Express.js):** Acts as middleware for processing logic. Using JavaScript for both Frontend and Backend reduces context-switching costs. The mysql2 library was chosen for its Promise API support, enabling safe async/await usage and efficient connection pool management.
3. **Data Tier (MySQL/InnoDB):** MySQL was chosen for its excellent support for the relational model, ACID transactions, Stored Procedures, and Triggers. The InnoDB engine plays a critical role with its row-level locking mechanism, preventing bottlenecks when multiple threads update race results simultaneously.

2.4 Database Server Configuration and Optimization

To support high concurrency, guarantee data persistence, and ensure smooth query execution under load, the MySQL database server was tuned with advanced configurations. These settings were applied in the `my.ini` (Windows) or `my.cnf` (Linux) configuration file.

2.4.1 RAM and Connection Tuning

RAM caching and connection limits are the primary drivers of database capacity:

- **InnoDB Buffer Pool Size** (`innodb_buffer_pool_size`):
 - *Purpose:* Caches data tables and indexes in RAM to minimize slow disk I/O.
 - *Setting:* `innodb_buffer_pool_size = 10G` (Allocated 10 GB from system RAM).

- *Verification query:*

```
1 SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
```

Expected output: 10737418240 (Bytes).

- **Maximum Connections** (max_connections):

- *Purpose:* Controls the maximum number of concurrent client connections, preventing the server from running out of file descriptors under heavy spikes.
- *Setting:* max_connections = 1000 (1 reserved for administrative root login).
- *Verification query:*

```
1 SHOW VARIABLES LIKE 'max_connections';
```

Expected output: 1000.

2.4.2 Hardware Resource Exploitation (CPU & SSD)

Tuning variables to fully leverage modern multi-core processors and fast SSD storage:

- **Buffer Pool Instances** (innodb_buffer_pool_instances):

- *Purpose:* Splits the large buffer pool into 8 separate regions to decrease mutex contention among CPU threads during concurrent operations.
- *Setting:* innodb_buffer_pool_instances = 8.
- *Verification query:*

```
1 SHOW VARIABLES LIKE 'innodb_buffer_pool_instances';
```

Expected output: 8.

- **SSD I/O Capacity** (innodb_io_capacity & innodb_io_capacity_max):

- *Purpose:* Calibrates the rate at which dirty pages are flushed to the solid-state drive. Increasing these numbers prevents I/O bottlenecks.
- *Setting:*

```
1 innodb_io_capacity = 2000
2 innodb_io_capacity_max = 4000
```

- *Verification query:*

```
1 SHOW VARIABLES LIKE 'innodb_io_capacity%';
```

Expected output: 2000 and 4000.

2.4.3 Auditing, Logging, and Resource Cleanup

Configuring logging mechanisms for security, auditing, performance diagnosis, and automated idle connection recycling:

- **General Query Log** (general_log):

- *Purpose:* Records all client connections and executed SQL statements for security auditing.

- *Setting*: `general_log = 1, general_log_file = "ASUSVIVOB00K14X.log"`.
- *Verification query*:

```
1 SHOW VARIABLES LIKE 'general_log%';
```

- **Slow Query Log (slow_query_log):**

- *Purpose*: Flags queries taking longer than a specified duration, useful for catching unindexed table scans.
- *Setting*: `slow_query_log = 1, slow_query_log_file = "ASUSVIVOB00K14X-slow.log", long_query_time = 2` (threshold set to 2 seconds).
- *Verification query*:

```
1 SHOW VARIABLES LIKE 'slow_query_log%';
```

- **Connection Cleanup Timers (wait_timeout & interactive_timeout):**

- *Purpose*: Cleans up orphaned or idle connections in a Sleep state after 120 seconds to free up connection slots.
- *Setting*: `wait_timeout = 120, interactive_timeout = 120`.
- *Verification query*:

```
1 SHOW VARIABLES LIKE 'wait_timeout';
```

2.4.4 Automated Database Backup Strategy (Backup Autopilot)

To prevent data loss from accidental drops or hardware failure, an automated backup rotation is configured. The backup strategy relies on a Windows batch script executed nightly via Windows Task Scheduler.

```
1 @echo off
2 :: Setup date variables for the backup filename
3 set backup_name=db_backup_%date:~10,4%-%date:~4,2%-%date:~7,2%.
  sql
4
5 :: Run mysqldump to export the database
6 "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" -u
  root -pPassword db_name > "C:\MySQL_Backups\%backup_name%"
7
8 :: Delete old backups (retains only the last 7 days of backups)
9 forfiles /p "C:\MySQL_Backups" /s /m *.sql /d -7 /c "cmd /c del
  @path"
```

Listing 2.1: Automated backup script — auto_backup.bat

2.4.5 Load Testing and System Validation

To validate the system's performance and connection stability under concurrent load, an API load testing suite was set up using **Artillery**.

- **Tooling**: Node.js Artillery testing utility.

- **Test Scenario** (test-tai.yml): Simulates concurrent requests targeted at the backend API endpoints over a sustained period of 60 seconds with an arrival rate of 20 virtual users per second.

```
1 config:
2   target: "http://localhost:8080"
3   phases:
4     - duration: 60
5       arrivalRate: 20
6 scenarios:
7   - name: "Customer Query Flow"
8     flow:
9       - get:
10         url: "/api/products"
```

Listing 2.2: Artillery load test configuration — test-tai.yml

- **Execution Command:**

```
1 artillery run test-tai.yml
```

- **Validation Criteria:**

1. Check active connection threads during load using: `SHOW STATUS LIKE 'Threads_connected';`.
2. Ensure zero HTTP 5xx responses and zero dropped socket connections (Errors: 0 in the Artillery summary).
3. Check the slow query log file to ensure that no queries exceeded the 2-second threshold.

Chapter 3

Installation & Deployment

This chapter describes in detail the database design process, SQL scripts (DDL, Triggers, Procedures, Views), and deployment steps.

3.1 Database Design and Normalization

The database schema was rigorously designed to achieve **BCNF (Boyce-Codd Normal Form)**, ensuring data integrity and completely eliminating redundancy.

3.1.1 Relational Schema

- CHAMPIONSHIPS(champ_code, name, description)
- TEAMS(team_code, name, brand, owner, description)
- DRIVERS(driver_code, name, date_of_birth, nationality)
- CONTRACTS(contract_id, driver_code*, team_code*, is_active)
- RACES(race_code, name, num_laps, start_time, champ_code*)
- RACE_ENTRIES(entry_id, race_code*, contract_id*)
- RESULTS(entry_id*, end_time, laps_completed, status, points)
- SPONSORS(sponsor_code, name, industry, description)
- TEAM_SPONSORSHIPS(sponsorship_id, team_code*, sponsor_code*, funding_amount, start_year, end_year)
- PENALTIES(penalty_id, entry_id*, type, severity_value, reason, is_applied)

(* denotes Foreign Key)

Detailed Explanation:

The system consists of 10 main entities (tables), designed to comprehensively manage the lifecycle of an F1 racing season.

- **CHAMPIONSHIPS (Seasons):** Stores information about racing seasons.
 - *Attributes:* champ_code (VARCHAR(10) - Season code - PK), name (VARCHAR(255) - Season name), description (TEXT - Description).
- **TEAMS (Racing Teams):** Manages information about participating teams.
 - *Attributes:* team_code (VARCHAR(10) - Team code - PK), name (VARCHAR(100) - Team name), brand (VARCHAR(100) - Car brand), owner

(VARCHAR(100) - Owner), description (TEXT - Description).

- **DRIVERS (Drivers):** Manages driver profile information.
 - *Attributes:* driver_code (VARCHAR(10) - Driver code - PK), name (VARCHAR(100) - Name), date_of_birth (DATE - Date of birth), nationality (VARCHAR(50) - Nationality), biography (TEXT - Biography).
- **CONTRACTS (Contracts):** Junction table storing the signing history between Drivers and Teams.
 - *Attributes:* contract_id (INT - Contract ID - PK), driver_code (VARCHAR(10) - Driver code - FK), team_code (VARCHAR(10) - Team code - FK), is_active (TINYINT - Active status).
- **RACES (Races):** Information about each race in a season.
 - *Attributes:* race_code (VARCHAR(10) - Race code - PK), name (VARCHAR(100) - Race name), num_laps (INT - Number of laps), location (VARCHAR(255) - Location), start_time (DATETIME(3) - Start time), champ_code (VARCHAR(10) - Season code - FK), description (TEXT - Description).
- **RACE_ENTRIES (Race Entries):** Junction table recording driver registration (via contract) for a specific race.
 - *Attributes:* entry_id (INT - Entry ID - PK), race_code (VARCHAR(10) - Race code - FK), contract_id (INT - Contract ID - FK).
- **RESULTS (Race Results):** Records driver performance after a race.
 - *Attributes:* entry_id (INT - Entry ID - PK/FK), end_time (DATETIME(3) - Finish time), laps_completed (INT - Laps completed), status (ENUM - Status Finished/DNF/Accident), points (INT - Points).
- **SPONSORS (Sponsors):** Manages list of sponsors.
 - *Attributes:* sponsor_code (VARCHAR(10) - Sponsor code - PK), name (VARCHAR(100) - Sponsor name), industry (VARCHAR(50) - Industry), description (TEXT - Description).
- **TEAM_SPONSORSHIPS (Team Sponsorships):** Junction table tracking funding history between sponsors and teams.
 - *Attributes:* sponsorship_id (INT - Sponsorship ID - PK), team_code (VARCHAR(10) - Team code - FK), sponsor_code (VARCHAR(10) - Sponsor code - FK), funding_amount (DECIMAL(15,2) - Funding amount), start_year (INT - Start year), end_year (INT - End year).
- **PENALTIES (Penalties):** Records driver penalties during races.
 - *Attributes:* penalty_id (INT - Penalty ID - PK), entry_id (INT - Entry ID - FK), type (VARCHAR(50) - Penalty type), severity_value (INT - Penalty severity value), reason (VARCHAR(255) - Reason), is_applied (TINYINT - Applied status).

Based on the foreign key structure, the entities in the system have the following relationships:

1. **Championship - Race (1:N):** A championship (CHAMPIONSHIPS) includes multiple races (RACES).
2. **Team - Contract (1:N):** A team (TEAMS) can sign multiple contracts (CONTRACTS) with different drivers.
3. **Driver - Contract (1:N):** A driver (DRIVERS) can have multiple contracts (CONTRACTS) with different teams throughout their career.
4. **Race - Entry (1:N):** Each race (RACES) has multiple race entries (RACE_ENTRIES).
5. **Contract - Entry (1:N):** A contract (CONTRACTS - representing 1 driver of 1 team) can be used to register for multiple races (RACE_ENTRIES).
6. **Entry - Result (1:1):** Each race entry (RACE_ENTRIES) has exactly one corresponding result record (RESULTS).
7. **Team - Sponsor (N:M):** Sponsors (SPONSORS) can sponsor multiple teams, and teams (TEAMS) can be sponsored by multiple sponsors, managed via TEAM_SPONSORSHIPS.
8. **Entry - Penalty (1:N):** A race entry (RACE_ENTRIES) can receive multiple penalties (PENALTIES).

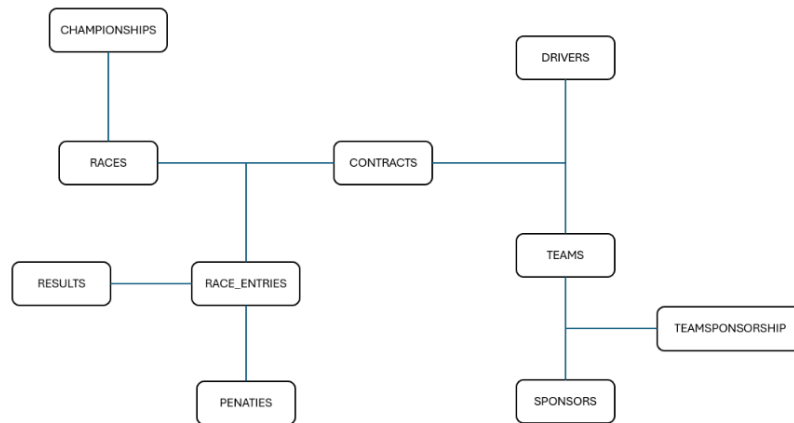


Figure 3.1: Entity-Relationship Diagram (ERD)

3.1.2 Normalization Assessment

- **First Normal Form (1NF):** All attributes are atomic. There are no repeating groups. For example: a team's driver list is managed through multiple records in **CONTRACTS** rather than using a comma-separated text string.
- **Second Normal Form (2NF):** All tables satisfy 1NF and have no partial dependencies. The **RACE_ENTRIES** table uses a surrogate key (**entry_id**) instead of a composite key, completely eliminating the possibility of 2NF violations.
- **Third Normal Form (3NF) & BCNF:** The schema eliminates all transitive dependencies. For instance, **team_name** does not appear in **RACE_ENTRIES**. Every determinant in the system is a superkey.

3.2 Data Definition Language (DDL) Script

The following SQL code initializes the database, table structures, and establishes foreign keys.

```
1 DROP DATABASE IF EXISTS F1_Championship_Management;
2 CREATE DATABASE F1_Championship_Management;
3 USE F1_Championship_Management;
4
5 SET SQL_SAFE_UPDATES = 0;
6
7 CREATE TABLE CHAMPIONSHIPS (
8     champ_code    VARCHAR(10)    PRIMARY KEY,
9     name          VARCHAR(255)   NOT NULL,
10    description    TEXT
11 );
12
13 CREATE TABLE TEAMS (
14     team_code     VARCHAR(10)    PRIMARY KEY,
15     name          VARCHAR(100)   NOT NULL,
16     brand         VARCHAR(100),
17     owner         VARCHAR(100),
18     description    TEXT
19 );
20
21 CREATE TABLE DRIVERS (
22     driver_code   VARCHAR(10)    PRIMARY KEY,
23     name          VARCHAR(100)   NOT NULL,
24     date_of_birth DATE,
25     nationality   VARCHAR(50),
26     biography     TEXT
27 );
28
29 CREATE TABLE CONTRACTS (
30     contract_id   INT            AUTO_INCREMENT PRIMARY KEY,
31     driver_code   VARCHAR(10),
32     team_code     VARCHAR(10),
33     is_active     TINYINT        DEFAULT 1,
34     FOREIGN KEY (driver_code) REFERENCES DRIVERS(driver_code),
35     FOREIGN KEY (team_code)   REFERENCES TEAMS(team_code)
36 );
37
38 CREATE TABLE RACES (
39     race_code     VARCHAR(10)    PRIMARY KEY,
40     name          VARCHAR(100)   NOT NULL,
41     num_laps      INT            NOT NULL,
42     location      VARCHAR(255),
43     start_time    DATETIME(3),
44     champ_code    VARCHAR(10),
45     description    TEXT,
46     FOREIGN KEY (champ_code) REFERENCES CHAMPIONSHIPS(
47         champ_code)
48 );
```

```

49 CREATE TABLE RACE_ENTRIES (
50     entry_id      INT AUTO_INCREMENT PRIMARY KEY,
51     race_code     VARCHAR(10),
52     contract_id   INT,
53     FOREIGN KEY (race_code) REFERENCES RACES(race_code),
54     FOREIGN KEY (contract_id) REFERENCES CONTRACTS(contract_id)
55     ,
56     UNIQUE(race_code, contract_id)
57 );
58
59 CREATE TABLE RESULTS (
60     entry_id      INT PRIMARY KEY,
61     end_time      DATETIME(3),
62     laps_completed INT,
63     status        ENUM('Finished', 'DNF', 'Accident') DEFAULT
64     'Finished',
65     points        INT DEFAULT 0,
66     FOREIGN KEY (entry_id) REFERENCES RACE_ENTRIES(entry_id),
67     CONSTRAINT chk_laps_completed CHECK (laps_completed >= 0),
68     CONSTRAINT chk_points          CHECK (points >= 0)
69 );
70
71 CREATE TABLE SPONSORS (
72     sponsor_code VARCHAR(10) PRIMARY KEY,
73     name          VARCHAR(100) NOT NULL,
74     industry      VARCHAR(50),
75     description   TEXT
76 );
77
78 CREATE TABLE TEAM_SPONSORSHIPS (
79     sponsorship_id INT AUTO_INCREMENT PRIMARY KEY,
80     team_code      VARCHAR(10),
81     sponsor_code   VARCHAR(10),
82     funding_amount DECIMAL(15,2),
83     start_year     INT,
84     end_year       INT,
85     FOREIGN KEY (team_code) REFERENCES TEAMS(team_code),
86     FOREIGN KEY (sponsor_code) REFERENCES SPONSORS(sponsor_code)
87 );
88
89 CREATE TABLE PENALTIES (
90     penalty_id     INT AUTO_INCREMENT PRIMARY KEY,
91     entry_id       INT,
92     type           VARCHAR(50),
93     severity_value INT,
94     reason         VARCHAR(255),
95     is_applied     TINYINT DEFAULT 1,
96     FOREIGN KEY (entry_id) REFERENCES RACE_ENTRIES(entry_id)
97 );

```

Listing 3.1: DDL — Table creation and constraint setup

3.3 Database Triggers for Business Logic

Embedding business logic at the database layer ensures that rules cannot be bypassed, regardless of whether data is entered via the API or directly through a DB admin tool.

3.3.1 Limiting the Number of Drivers per Team

Purpose: Prevent the registration of a 3rd driver from the same team in the same race.

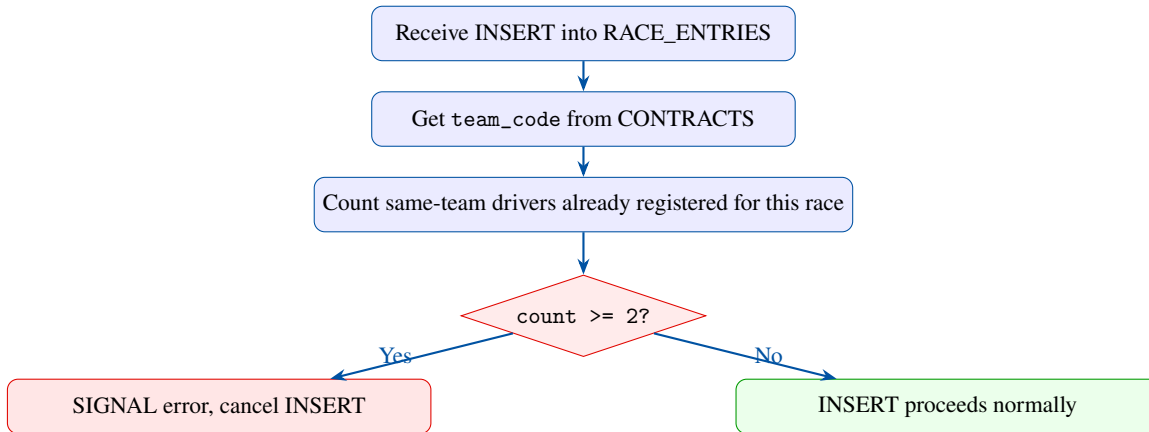


Figure 3.2: Processing flow of Trigger trg_limit_2_riders

This trigger ensures no team registers more than 2 drivers in the same race.

```

1 DELIMITER //
2 CREATE TRIGGER trg_limit_2_riders
3 BEFORE INSERT ON RACE_ENTRIES
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_team_id VARCHAR(10);
7     DECLARE v_count INT;
8
9     -- Lay ma doi cua tay dua sap dang ky
10    SELECT team_code INTO v_team_id
11    FROM CONTRACTS WHERE contract_id = NEW.contract_id;
12
13    -- Dem so tay dua cua doi nay da dang ky trong chang
14    SELECT COUNT(*) INTO v_count
15    FROM RACE_ENTRIES re
16    JOIN CONTRACTS c ON re.contract_id = c.contract_id
17    WHERE re.race_code = NEW.race_code
18           AND c.team_code = v_team_id;
19
20    -- Huy giao dich neu vuot qua so luong cho phép
21    IF v_count >= 2 THEN
22        SIGNAL SQLSTATE '45000'
23        SET MESSAGE_TEXT = 'Error: Each team can register a
24                               maximum of 2 drivers!';
25    END IF;
26 END //
27 DELIMITER ;
  
```

Listing 3.2: Trigger — trg_limit_2_riders

3.3.2 Time Validity Check

Purpose: Ensure `end_time > start_time` of the race. Applied to both INSERT and UPDATE on the RESULTS table. This trigger ensures the finish time occurs after the race start time.

```

1 DELIMITER //
2 CREATE TRIGGER trg_check_time_insert
3 BEFORE INSERT ON RESULTS
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_start_time DATETIME(3);
7     IF NEW.status = 'Finished' AND NEW.end_time IS NOT NULL
8     THEN
9         SELECT r.start_time INTO v_start_time
10        FROM RACE_ENTRIES re
11       JOIN RACES r ON re.race_code = r.race_code
12      WHERE re.entry_id = NEW.entry_id;
13
14     IF NEW.end_time <= v_start_time THEN
15         SIGNAL SQLSTATE '45000'
16         SET MESSAGE_TEXT =
17             'Error: Finish time must be greater than start
18             time!';
19     END IF;
20 END IF;
21 END //
22 DELIMITER ;

```

Listing 3.3: Trigger — `trg_check_time_insert`

3.3.3 Other Business Integrity Triggers

To achieve complete data integrity across all 10 tables, the remaining 8 triggers enforce additional validation rules at the database level:

```

1 -- Trigger 3: Time validity check on UPDATE
2 DELIMITER //
3 CREATE TRIGGER trg_check_time_update
4 BEFORE UPDATE ON RESULTS
5 FOR EACH ROW
6 BEGIN
7     DECLARE v_start_time DATETIME(3);
8     IF NEW.status = 'Finished' AND NEW.end_time IS NOT NULL
9     THEN
10        SELECT r.start_time INTO v_start_time
11       FROM RACE_ENTRIES re JOIN RACES r ON re.race_code = r.
12      race_code
13     WHERE re.entry_id = NEW.entry_id;
14     IF NEW.end_time <= v_start_time THEN
15         SIGNAL SQLSTATE '45000'
16         SET MESSAGE_TEXT = 'Error: Finish time must be
17         greater than start time!';
18     END IF;
19 END IF;

```

```

16     END IF;
17 END //
18
19 -- Trigger 4: Laps completed check on INSERT
20 CREATE TRIGGER trg_limit_laps_completed_insert
21 BEFORE INSERT ON RESULTS
22 FOR EACH ROW
23 BEGIN
24     DECLARE v_max_laps INT;
25     SELECT r.num_laps INTO v_max_laps
26     FROM RACE_ENTRIES re JOIN RACES r ON re.race_code = r.
27         race_code
28     WHERE re.entry_id = NEW.entry_id;
29     IF NEW.laps_completed > v_max_laps THEN
30         SIGNAL SQLSTATE '45000'
31         SET MESSAGE_TEXT = 'Error: Completed laps cannot exceed
32             total race laps!';
33     END IF;
34 END //
35
36 -- Trigger 5: Laps completed check on UPDATE
37 CREATE TRIGGER trg_limit_laps_completed_update
38 BEFORE UPDATE ON RESULTS
39 FOR EACH ROW
40 BEGIN
41     DECLARE v_max_laps INT;
42     SELECT r.num_laps INTO v_max_laps
43     FROM RACE_ENTRIES re JOIN RACES r ON re.race_code = r.
44         race_code
45     WHERE re.entry_id = NEW.entry_id;
46     IF NEW.laps_completed > v_max_laps THEN
47         SIGNAL SQLSTATE '45000'
48         SET MESSAGE_TEXT = 'Error: Completed laps cannot exceed
49             total race laps!';
50     END IF;
51 END //
52
53 -- Trigger 6: Prevent multiple active contracts
54 CREATE TRIGGER trg_single_active_contract
55 BEFORE INSERT ON CONTRACTS
56 FOR EACH ROW
57 BEGIN
58     DECLARE v_active_count INT;
59     IF NEW.is_active = 1 THEN
60         SELECT COUNT(*) INTO v_active_count FROM CONTRACTS
61         WHERE driver_code = NEW.driver_code AND is_active = 1;
62         IF v_active_count > 0 THEN
63             SIGNAL SQLSTATE '45000'
64             SET MESSAGE_TEXT = 'Error: A driver cannot have
65                 more than 1 active contract!';
66         END IF;
67     END IF;
68 END //

```

```

64
65 -- Trigger 7: Prevent negative penalty values
66 CREATE TRIGGER trg_validate_penalty_value
67 BEFORE INSERT ON PENALTIES
68 FOR EACH ROW
69 BEGIN
70     IF NEW.severity_value < 0 THEN
71         SIGNAL SQLSTATE '45000'
72         SET MESSAGE_TEXT = 'Error: Penalty severity value
73             cannot be negative!';
74     END IF;
75 END //
76
77 -- Trigger 8: Prevent future date of birth
78 CREATE TRIGGER trg_check_dob_insert
79 BEFORE INSERT ON DRIVERS
80 FOR EACH ROW
81 BEGIN
82     IF NEW.date_of_birth > CURDATE() THEN
83         SIGNAL SQLSTATE '45000'
84         SET MESSAGE_TEXT = 'Error: Date of birth cannot be in
85             the future!';
86     END IF;
87 END //
88
89 -- Trigger 9: Prevent non-positive race laps
90 CREATE TRIGGER trg_prevent_negative_laps_race
91 BEFORE INSERT ON RACES
92 FOR EACH ROW
93 BEGIN
94     IF NEW.num_laps <= 0 THEN
95         SIGNAL SQLSTATE '45000'
96         SET MESSAGE_TEXT = 'Error: Total race laps must be
97             greater than 0!';
98     END IF;
99 END //
100
101 -- Trigger 10: Validate sponsorship years
102 CREATE TRIGGER trg_check_sponsorship_years
103 BEFORE INSERT ON TEAM_SPONSORSHIPS
104 FOR EACH ROW
105 BEGIN
106     IF NEW.start_year > NEW.end_year THEN
107         SIGNAL SQLSTATE '45000'
108         SET MESSAGE_TEXT = 'Error: Sponsorship start year
109             cannot be greater than end year!';
110     END IF;
111 END //
112 DELIMITER ;

```

Listing 3.4: Triggers — Data integrity validation triggers

Rationale for database-level constraints: If validation is only performed at the Frontend or Backend, users can bypass it using Postman or MySQL Workbench. Triggers ensure constraints

are enforced at **EVERY ACCESS POINT**, forming a strong secondary layer of defense.

3.4 Stored Procedure and Transaction

Purpose: Automatically calculate F1 points based on time ranking, wrapped in an ACID Transaction.

F1 point calculation requires ranking drivers by their finish times. Performing this dynamically via a View would degrade performance. Therefore, a Stored Procedure is called immediately after saving results. This process runs inside a Transaction to ensure Atomicity.

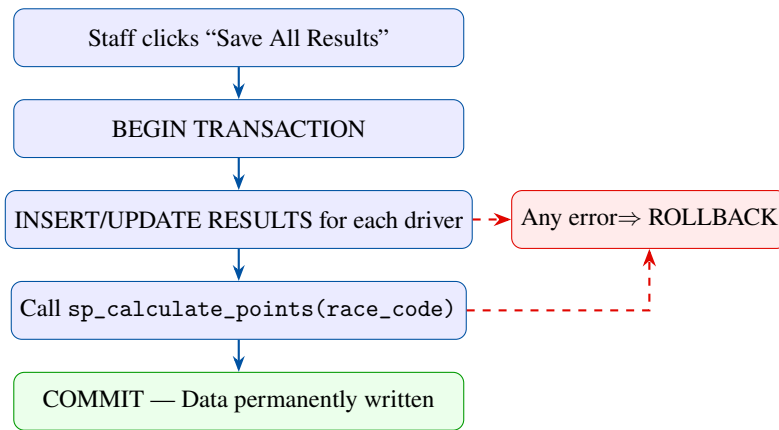


Figure 3.3: Transaction flow of Module 2 — Result Entry

```

1 DELIMITER //
2 CREATE PROCEDURE sp_calculate_points(IN p_race_code VARCHAR(10)
3 )
4 BEGIN
5     DECLARE EXIT HANDLER FOR SQLEXCEPTION
6     BEGIN
7         ROLLBACK;
8         RESIGNAL;
9     END;
10    START TRANSACTION;
11
12    -- Buoc 1: Reset diem chang ve 0
13    UPDATE RESULTS res
14    JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
15    SET res.points = 0
16    WHERE re.race_code = p_race_code;
17
18    -- Buoc 2: Xep hang thoi gian va cap nhat diem
19    UPDATE RESULTS target
20    JOIN (
21        SELECT res.entry_id,
22        RANK() OVER (
23            ORDER BY TIMESTAMPDIFF(MICROSECOND, r.start_time,
24                res.end_time) ASC
25        ) AS pos
26        FROM RESULTS res
27        JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
28        JOIN RACES r ON re.race_code = r.race_code
29        WHERE re.race_code = p_race_code AND res.status = '
30        Finished'
31    ) rnk ON target.entry_id = rnk.entry_id
32    SET target.points = CASE
33        WHEN rnk.pos = 1 THEN 25 WHEN rnk.pos = 2 THEN 18
34        WHEN rnk.pos = 3 THEN 15 WHEN rnk.pos = 4 THEN 12
35        WHEN rnk.pos = 5 THEN 10 WHEN rnk.pos = 6 THEN 8
36        WHEN rnk.pos = 7 THEN 6 WHEN rnk.pos = 8 THEN 4
37        WHEN rnk.pos = 9 THEN 2 WHEN rnk.pos = 10 THEN 1
38        ELSE 0

```

```

37     END;
38
39     COMMIT;
40 END //
41 DELIMITER ;

```

Listing 3.5: Stored Procedure sp_calculate_points — Automatic F1 point calculation

3.4.1 Operational Stored Procedures

To support core CRUD operations, enforce business rules, and optimize data access from the Backend, the following 9 Stored Procedures are implemented:

```

1  -- Procedure 2: Add a new driver with birthdate validation
2  DELIMITER //
3  CREATE PROCEDURE sp_add_driver(
4      IN p_driver_code VARCHAR(10),
5      IN p_name VARCHAR(100),
6      IN p_dob DATE,
7      IN p_nationality VARCHAR(50),
8      IN p_biography TEXT
9  )
10 BEGIN
11     IF p_dob > CURDATE() THEN
12         SIGNAL SQLSTATE '45000'
13         SET MESSAGE_TEXT = 'Error: Date of birth cannot be in
14             the future!';
15     ELSE
16         INSERT INTO DRIVERS VALUES (p_driver_code, p_name,
17             p_dob, p_nationality, p_biography);
18     END IF;
19 END //
20
21 -- Procedure 3: Create a new contract and automatically manage
22 -- active status
23 CREATE PROCEDURE sp_create_contract(
24     IN p_driver_code VARCHAR(10),
25     IN p_team_code VARCHAR(10),
26     IN p_is_active TINYINT
27 )
28 BEGIN
29     IF p_is_active = 1 THEN
30         UPDATE CONTRACTS SET is_active = 0 WHERE driver_code =
31             p_driver_code AND is_active = 1;
32     END IF;
33     INSERT INTO CONTRACTS (driver_code, team_code, is_active)
34     VALUES (p_driver_code, p_team_code, p_is_active);
35 END //
36
37 -- Procedure 4: Register driver for a race
38 CREATE PROCEDURE sp_register_race_entry(
39     IN p_race_code VARCHAR(10),
40     IN p_contract_id INT
41 )

```

```

37 BEGIN
38     INSERT INTO RACE_ENTRIES (race_code, contract_id) VALUES (
39         p_race_code, p_contract_id);
40
41 -- Procedure 5: Add a new penalty
42 CREATE PROCEDURE sp_add_penalty(
43     IN p_entry_id INT,
44     IN p_type VARCHAR(50),
45     IN p_severity INT,
46     IN p_reason VARCHAR(255)
47 )
48 BEGIN
49     INSERT INTO PENALTIES (entry_id, type, severity_value,
50         reason, is_applied)
51     VALUES (p_entry_id, p_type, p_severity, p_reason, 1);
52 END //
53
54 -- Procedure 6: Add a new sponsor
55 CREATE PROCEDURE sp_add_sponsor(
56     IN p_sponsor_code VARCHAR(10),
57     IN p_name VARCHAR(100),
58     IN p_industry VARCHAR(50),
59     IN p_description TEXT
60 )
61 BEGIN
62     INSERT INTO SPONSORS VALUES (p_sponsor_code, p_name,
63         p_industry, p_description);
64 END //
65
66 -- Procedure 7: Retrieve driver performance summary
67 CREATE PROCEDURE sp_get_driver_performance(IN p_driver_code
68     VARCHAR(10))
69 BEGIN
70     SELECT
71         d.name,
72         COUNT(res.entry_id) AS total_races,
73         SUM(res.points) AS total_points,
74         SUM(CASE WHEN res.points = 25 THEN 1 ELSE 0 END) AS
75             total_wins,
76         SUM(CASE WHEN res.status = 'DNF' THEN 1 ELSE 0 END) AS
77             total_dnfs
78     FROM DRIVERS d
79     LEFT JOIN CONTRACTS c ON d.driver_code = c.driver_code
80     LEFT JOIN RACE_ENTRIES re ON c.contract_id = re.contract_id
81     LEFT JOIN RESULTS res ON re.entry_id = res.entry_id
82     WHERE d.driver_code = p_driver_code
83     GROUP BY d.driver_code, d.name;
84 END //
85
86 -- Procedure 8: Get drivers for a team ordered by active status
87 CREATE PROCEDURE sp_get_team_drivers(IN p_team_code VARCHAR(10)
88 )

```

```

83 BEGIN
84     SELECT DISTINCT d.driver_code, d.name, d.nationality, c.
           is_active
85     FROM DRIVERS d
86     JOIN CONTRACTS c ON d.driver_code = c.driver_code
87     WHERE c.team_code = p_team_code
88     ORDER BY c.is_active DESC, d.name ASC;
89 END //
90
91 -- Procedure 9: Terminate all active contracts for a driver
92 CREATE PROCEDURE sp_terminate_active_contracts(IN p_driver_code
           VARCHAR(10))
93 BEGIN
94     UPDATE CONTRACTS SET is_active = 0 WHERE driver_code =
           p_driver_code AND is_active = 1;
95 END //
96
97 -- Procedure 10: Transfer driver to a new team
98 CREATE PROCEDURE sp_transfer_driver(
99     IN p_driver_code VARCHAR(10),
100    IN p_new_team_code VARCHAR(10)
101 )
102 BEGIN
103     CALL sp_terminate_active_contracts(p_driver_code);
104     CALL sp_create_contract(p_driver_code, p_new_team_code, 1);
105 END //
106 DELIMITER ;

```

Listing 3.6: Stored Procedures — Driver management and query procedures

3.5 Views and Indexing

Views are used to encapsulate complex JOIN queries and aggregate functions, allowing the Application tier to use simple SELECT statements.

```

1  -- View 1: v_race_performance
2  CREATE VIEW v_race_performance AS
3  SELECT re.race_code, c.driver_code, c.team_code, res.status,
4         res.laps_completed, res.points,
5         TIMESTAMPDIFF(MICROSECOND, r.start_time, res.end_time) /
           1000000 AS finish_time_seconds,
6         r.start_time AS race_start_time,
7         r.champ_code
8  FROM RESULTS res
9  JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
10 JOIN RACES r ON re.race_code = r.race_code
11 JOIN CONTRACTS c ON re.contract_id = c.contract_id;
12
13 -- View 2: v_driver_standings
14 CREATE VIEW v_driver_standings AS
15 SELECT
16     d.driver_code, d.name, d.nationality, t.name AS team_name,
17     SUM(v.points) AS total_score,

```

```

18      SUM(CASE WHEN v.status = 'Finished' THEN v.
        finish_time_seconds ELSE 0 END) AS total_season_time
19 FROM DRIVERS d
20 JOIN v_race_performance v ON v.driver_code = d.driver_code
21 JOIN TEAMS t ON v.team_code = t.team_code
22 WHERE v.champ_code = (SELECT champ_code FROM CHAMPIONSHIPS
        ORDER BY champ_code DESC LIMIT 1)
23 GROUP BY d.driver_code, d.name, d.nationality, t.name
24 ORDER BY total_score DESC, total_season_time ASC;
25
26 -- View 3: v_team_standings
27 CREATE VIEW v_team_standings AS
28 SELECT
29     t.team_code, t.name AS team_name, t.brand,
30     SUM(v.points) AS team_total_score,
31     SUM(CASE WHEN v.status = 'Finished' THEN v.
        finish_time_seconds ELSE 0 END) AS team_total_time
32 FROM TEAMS t
33 JOIN v_race_performance v ON v.team_code = t.team_code
34 WHERE v.champ_code = (SELECT champ_code FROM CHAMPIONSHIPS
        ORDER BY champ_code DESC LIMIT 1)
35 GROUP BY t.team_code, t.name, t.brand
36 ORDER BY team_total_score DESC, team_total_time ASC;
37
38 -- View 4: v_active_contracts
39 CREATE VIEW v_active_contracts AS
40 SELECT c.contract_id, d.driver_code, d.name AS driver_name, t.
        team_code, t.name AS team_name
41 FROM CONTRACTS c
42 JOIN DRIVERS d ON c.driver_code = d.driver_code
43 JOIN TEAMS t ON c.team_code = t.team_code
44 WHERE c.is_active = 1;
45
46 -- View 5: v_race_schedule
47 CREATE VIEW v_race_schedule AS
48 SELECT r.race_code, r.name AS race_name, ch.name AS
        championship_name,
49     r.location, r.start_time
50 FROM RACES r
51 JOIN CHAMPIONSHIPS ch ON r.champ_code = ch.champ_code;
52
53 -- View 6: v_team_sponsors
54 CREATE VIEW v_team_sponsors AS
55 SELECT t.team_code, t.name AS team_name, s.name AS sponsor_name
        , s.industry, ts.funding_amount
56 FROM TEAM_SPONSORSHIPS ts
57 JOIN TEAMS t ON ts.team_code = t.team_code
58 JOIN SPONSORS s ON ts.sponsor_code = s.sponsor_code;
59
60 -- View 7: v_race_penalties
61 CREATE VIEW v_race_penalties AS
62 SELECT r.race_code, r.name AS race_name, d.name AS driver_name,
        t.name AS team_name,

```

```

63      p.type AS penalty_type, p.severity_value, p.reason
64 FROM PENALTIES p
65 JOIN RACE_ENTRIES re ON p.entry_id = re.entry_id
66 JOIN RACES r ON re.race_code = r.race_code
67 JOIN CONTRACTS c ON re.contract_id = c.contract_id
68 JOIN DRIVERS d ON c.driver_code = d.driver_code
69 JOIN TEAMS t ON c.team_code = t.team_code;
70
71 -- View 8: v_driver_penalties_summary
72 CREATE VIEW v_driver_penalties_summary AS
73 SELECT d.driver_code, d.name AS driver_name, COUNT(p.penalty_id
74        ) AS total_penalties, SUM(p.severity_value) AS
75        total_severity
76 FROM PENALTIES p
77 JOIN RACE_ENTRIES re ON p.entry_id = re.entry_id
78 JOIN CONTRACTS c ON re.contract_id = c.contract_id
79 JOIN DRIVERS d ON c.driver_code = d.driver_code
80 GROUP BY d.driver_code, d.name;
81
82 -- View 9: v_driver_career_summary
83 CREATE VIEW v_driver_career_summary AS
84 SELECT d.driver_code, d.name, d.nationality,
85        COUNT(DISTINCT re.race_code) AS total_races_entered,
86        SUM(res.points) AS total_career_points,
87        SUM(CASE WHEN res.status = 'Finished' THEN 1 ELSE 0 END)
88        AS total_finishes
89 FROM DRIVERS d
90 LEFT JOIN CONTRACTS c ON d.driver_code = c.driver_code
91 LEFT JOIN RACE_ENTRIES re ON c.contract_id = re.contract_id
92 LEFT JOIN RESULTS res ON re.entry_id = res.entry_id
93 GROUP BY d.driver_code, d.name, d.nationality;
94
95 -- View 10: v_championship_summary
96 CREATE VIEW v_championship_summary AS
97 SELECT ch.champ_code, ch.name AS championship_name,
98        COUNT(DISTINCT r.race_code) AS total_races,
99        COUNT(DISTINCT c.team_code) AS total_participating_teams
100        ,
101        COUNT(DISTINCT c.driver_code) AS
102        total_participating_drivers
103 FROM CHAMPIONSHIPS ch
104 LEFT JOIN RACES r ON ch.champ_code = r.champ_code
105 LEFT JOIN RACE_ENTRIES re ON r.race_code = re.race_code
106 LEFT JOIN CONTRACTS c ON re.contract_id = c.contract_id
107 GROUP BY ch.champ_code, ch.name;

```

Listing 3.7: Views — All 10 views for reporting and queries

To optimize queries scanning thousands of records, the following 10 performance indexes are established in the database:

```

1 CREATE INDEX idx_race_code          ON RACE_ENTRIES(race_code
2    );
3 CREATE INDEX idx_team_code_contracts ON CONTRACTS(team_code,
4    driver_code);

```

```
3 CREATE INDEX idx_results_status_time ON RESULTS(status ,
    end_time);
4 CREATE INDEX idx_races_start_time ON RACES(start_time);
5 CREATE INDEX idx_races_champ_code ON RACES(champ_code);
6 CREATE INDEX idx_drivers_nationality ON DRIVERS(nationality);
7 CREATE INDEX idx_contracts_is_active ON CONTRACTS(is_active);
8 CREATE INDEX idx_drivers_name ON DRIVERS(name);
9 CREATE INDEX idx_penalties_entry ON PENALTIES(entry_id);
10 CREATE INDEX idx_team_sponsorships_yr ON TEAM_SPONSORSHIPS(
    start_year, end_year);
```

Listing 3.8: Performance optimization with Indexes

3.6 Deployment Guide

3.6.1 Step 1: Initialize the Database

Clone the project repository to your machine. Open MySQL terminal or MySQL Workbench, log in with an admin account and run the SQL file:

```
1 mysql -u root -p
2 # Nhập mật khẩu khi được yêu cầu
3 SOURCE /path/to/project/database/schema.sql;
```

Verify with the command: `USE F1_Championship_Management; SHOW TABLES;`

3.6.2 Step 2: Configure Backend

Navigate to the backend directory. Requires Node.js (v18+) installed.

```
1 cd backend
2 npm install
```

Create a `.env` file in the root of the backend directory with the following content:

```
1 DB_HOST=localhost
2 DB_PORT=3306
3 DB_USER=root
4 DB_PASSWORD=your_password
5 DB_NAME=F1_Championship_Management
6 PORT=3000
```

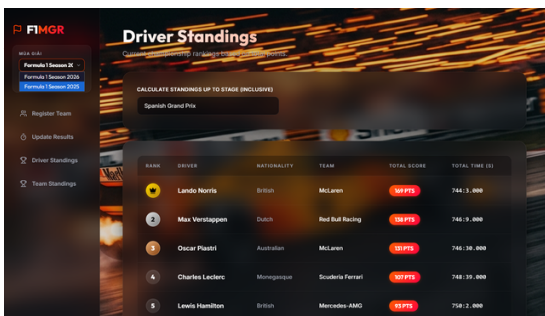
Start the Express server: `npm run dev`.

3.6.3 Step 3: Launch Frontend

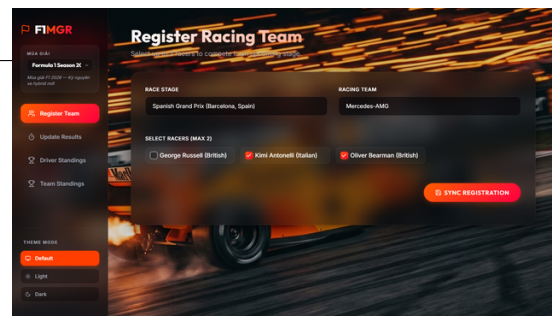
Open a new terminal and navigate to the frontend directory.

```
1 cd frontend
2 npm install
3 npm run dev
```

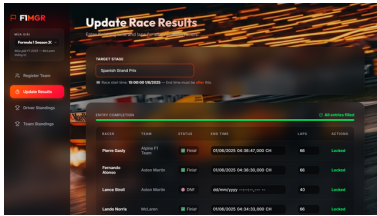
The application will run at `http://localhost:5173`. You can access the “Standings” module to verify the standings feature, or the “Update Results” module to test Transaction and Stored Procedure flows.



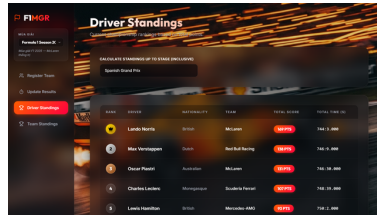
(a) Season management interface



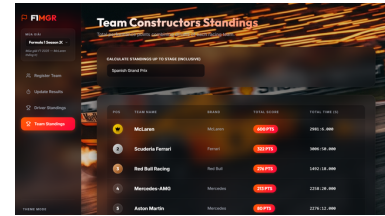
(b) Driver registration interface



(c) Result update interface



(d) Driver standings interface



(e) Constructor standings interface

Figure 3.4: F1 Championship Management System interface

3.7 Results

Full project source code: <https://github.com/PhamNhatMinh21/DBMS>

Chapter 4

Conclusion

4.1 Summary of Achievements

The development and deployment of the F1 Championship Management System has demonstrated the practical application of advanced DBMS concepts. The completed system achieved the following:

- A database schema of 10 tables achieving BCNF, thoroughly resolving insertion, update, and deletion anomalies.
- 9 Foreign Keys established to ensure referential integrity.
- 10 Database Triggers strictly enforcing business rules (driver limits, time validity, and formatting) completely independent of the application layer.
- Centralized business logic and automated calculations through 10 Stored Procedures (including F1 points calculation utilizing the `RANK()` function) running within safe, atomicity-guaranteed ACID Transactions.
- Data abstraction and reporting through 10 Views, combined with 10 Indexes optimizing retrieval speed.

4.2 Lessons Learned

The project reinforced several important principles in database design:

1. **Database as the Single Source of Truth:** Embedding the 2-driver limit logic directly into a MySQL Trigger permanently protects the system from API client errors.
2. **Transaction Scope:** When simultaneously updating results for 20 drivers, a Transaction is mandatory. Without `START TRANSACTION` and `COMMIT/ROLLBACK`, a single mid-process error would corrupt the entire standings.
3. **Normalize First, Code Later:** Separating the `CONTRACTS` table instead of embedding `team_code` directly into `DRIVERS` prevented countless data synchronization bugs later on.

4.3 Future Development

Although fully functional, the system can be further extended:

- **Handling Race Conditions:** Currently, the Trigger counts drivers using `SELECT COUNT`. Under extremely high concurrent load, race conditions may occur. An upgrade to `SELECT ... FOR UPDATE` should be implemented to lock records during processing.

- **Detailed Contract History Tracking:** Upgrade the CONTRACTS table by adding start_date and end_date instead of using only the is_active flag, enabling full contract lifecycle retrieval.
- **System Security:** Integrate Role-Based Access Control (RBAC) with JSON Web Tokens (JWT) at the Node.js layer to restrict result writing permissions to organizers only.
- **Audit Logging:** Create an AUDIT_LOG table and set up AFTER UPDATE triggers to precisely record who modified sensitive result data and when.

References

- [1] Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
- [2] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [3] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] MariaDB Foundation. (2024). *MariaDB Server Documentation*. Retrieved from <https://mariadb.com/kb/en/documentation/>
- [5] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [6] Fédération Internationale de l'Automobile (FIA). (2024). *Formula 1 Sporting Regulations*. Retrieved from <https://www.fia.com/>